

ユニット 3・第3-4週

ファイルとファイルシステム

📅 第3-4週・日程は未定 📄 演習 TD2（解答 + 生成AIチャレンジ） 📺 アニメーション 4 本

マニュアル M3・ユニットの理論

ファイルとファイルシステム

ユニット 3 の基本の読み物（約 1.5 時間）。各節の最後に**実践** →リンクがあります。出力はすべて実物で、コースのマシン（Ubuntu 26.04、Linux 7.0、NVMe 上の ext4）で TD2 のプログラムを実行して得たものです。

目次

- | | | | |
|----|------------------|----|---|
| §1 | 2 つの層：ディスクとカーネル | §5 | 特殊ファイル：FIFO、デバイス、 <code>/proc</code> |
| §2 | inode：メタデータとブロック | §6 | ディレクトリの走査 |
| §3 | 名前：ディレクトリ、リンク、パス | §7 | その先へ：ジャーナリング、
コピーオンライト、 <code>fsync</code> |
| §4 | 開いているファイル：3 つの表 | | |

§1 2 つの層：ディスクとカーネル

ファイルは 2 つの場所に存在します。**ディスク上**では、ファイルシステムが巨大なブロックの配列（コースのマシンでは 4 KiB）を管理します：全体を記述するスーパーブロック、空いているブロックと inode を示すビットマップ、*inode* 表、そしてデータブロック。ext4 はこの構造をパーティション全体でブロックグループとして繰り返し、スーパーブロックのコピーも置きます。inode の数はフォーマット時に固定されます：

実際の出力

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /
block size: 4096 bytes
```

カーネル内では、プロセスがファイルを使っている間、3 つの表があります：各プロセスのディスクリプタ表、システムのオープンファイル表、メモリ上の inode 表 (§4)。さらにその上で、ページキャッシュが最近読んだブロックを RAM に保持します：だからファイルを 2 回目に読むとき、ほとんどディスクに触れないのです。

スライド P3 で見える →

§2 inode：メタデータとブロック

Unix はファイルの**内容**と**メタデータ**を分けています。メタデータは番号で識別される **inode** にあります：種類と権限（`st_mode`）、ハードリンク数（`st_nlink`）、所有者とグループ、サイズ、デバイス、タイムスタンプ、ブロックの一覧。inode に名前は**含まれません**。`stat`、`fstat`（ディスクリプタに対して）、`lstat`（リンクをたどらない）で読みます：

実際の出力

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096   regular file
Device: 259,5      Inode: 12190686    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: ( 1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

4つのタイムスタンプ：**Access**（最後の読み込み。Linux は `relatime` で遅延して更新）、**Modify**（内容の最後の変更）、**Change**（*inode* の最後の変更：権限、リンク、所有者...または内容）、**Birth**（作成。ext4 にあり `statx` で読む）。`ctime` を作成時刻だと思うのはよくある誤りです（元のスライドもそうでした）。

種類は `st_mode` に対して `S_ISREG`、`S_ISDIR`、`S_ISLNK`、`S_ISCHR`、`S_ISBLK`、`S_ISFIFO`、`S_ISSOCK` マクロで調べます。`stat` の典型的なエラーは `ENOENT`（存在しない）、`EACCES`（パス上のディレクトリに検索権 `x` がない）、`ENOTDIR`、`ELoop` です。

ブロックはどこに？ ext2 と ext3 は inode に 15 個のポインタを持っていました：直接 12 個、一重間接（ポインタで埋まったブロック）1 個、二重 1 個、三重 1 個。4 KiB ブロックと 4 バイトのポインタなら 1 ブロックに 1024 個入り、約 4 TiB まで扱えますが、遠い位置では最大 3 つの索引ブロックを読む必要があります。ext4 は**エクステンツ**を使います：（論理開始位置、長さ ≤ 32,768 ブロック、物理開始位置）で表す連続領域です。inode に 4 つ入り、それ以上必要なら木を作ります。TD2 が書いた 10 MB のファイルは 2 つだけで済みます：

実際の出力

```
$ head -c 10000000 /dev/urandom > big.bin; sync big.bin; filefrag -v big.bin
Filesystem type is: ef53
File size of big.bin is 10000000 (2442 blocks of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
  0:         0..       2047:   15654912..   15656959:    2048:
  1:      2048..       2441:   15673843..   15674236:    394:   15656960: last, eof
big.bin: 2 extents found
```

実践 → A3.4 inode からブロックへ・TD2・1

§3 名前：ディレクトリ、リンク、パス

ディレクトリは、データが **名前** → **inode** 番号の組のリストであるファイルで、**.**（自分自身）と **..**（親）も含まれます。 `/home/student/notas.txt` を開くとき、カーネルはルートの inode（ext4 では 2 番）から始め、ディレクトリを読んで `home` を探し、その inode に進み...と要素ごとに続けます（`path_resolution(7)`）。各要素ではそのディレクトリへの検索権（`x`）が必要です。Linux は最近の解決結果を `dentry` キャッシュに保持します。

ハードリンク（`link`、`ln`）は**同じ** inode を指す別のエントリです：どちらの名前も同じく「本物」で、`st_nlink` がそれを数えます。ディレクトリには作れず（木が壊れる）、ファイルシステムもまたげません（inode 番号はローカル）。**シンボリックリンク**（`symlink`、`ln -s`）は内容がパスである新しい inode です。解決時にたどられ（最大 40 回、それを超えると `ELOOP`）、指す先が消えると宙に浮きます。

実際の出力

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student  9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

`rm` は `unlink` を呼び、**エントリ**を1つ消して `st_nlink` から1を引きます。カーネルが inode とそのブロックを解放するのは、`st_nlink = 0` **かつ**どのプロセスも開いていないときだけです。それまでは、ディスクリプタ経由でまだ読めます：

実際の出力

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l
atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

⚠ 確認してから使う：TOCTOU 競合

`if (access(path, W_OK) == 0) fd = open(path, O_WRONLY);` には隙間があります：2 つの呼び出しの間に、別のプロセスが `path` を他のファイルへのリンクにすり替えられるのです。直接開いてエラー（`EACCES`、`ENOENT`）を処理するか、`O_NOFOLLOW` や `RESOLVE_NO_SYMLINKS` 付きの `openat2()` を使いましょう。また `access` は実効 UID ではなく実 UID で確認します。

実践 → A3.1 inode とディレクトリ

§4 開いているファイル：3つの表

`open(path, flags, mode)` はパスを解決し、権限を確認し、**空いている最小のディスクリプタ**を返します。フラグ：アクセス（`O_RDONLY`、`O_WRONLY`、`O_RDWR`）、作成（`O_CREAT`、`O_EXCL`、`O_TRUNC`）。要求した権限は `umask` を通る）、状態（`O_APPEND`、`O_NONBLOCK`、`O_CLOEXEC`）。glibc は `open` を `openat(AT_FDCWD, ...)` で実装しており、`strace` に見えるのはこちらです。

各ディスクリプタの裏には3つの層があります：

1. **ディスクリプタ表**（プロセスごとに1つ）：番号（0、1、2、3...）はこの表の添字です。
2. **オープンファイル表**（システム全体で1つ）：`open` のたびに、モード、現在の**オフセット**、参照カウンタを持つエントリが作られます。
3. **メモリ上の inode 表**：ファイルごとに inode のコピーが1つあり、すべての `open` で共有されます。だから誰が各エントリを共有するかが重要です。同じファイルへの2回の `open` は、独立したオフセットを持つ2つのエントリを作ります。`dup`、`dup2`、`fork` はポインタをコピーし、オフセットを共有します。`donnees.txt` = `abcdefgh` の場合：

実際の出力

```
$ ./alias_open
abcdabcd
```

実際の出力

```
$ ./alias_dup
abcdefgh
```

実際への影響：`./prog > out 2> out` は `out` を2回開くので、エラー出力が標準出力を上書きします。`./prog > out 2>&1` は `dup2` を使うので何も失いません。`O_APPEND` なら各 `write` は原子的に末尾へ移動します：サーバーはこうしてログを書きます。`close` は参照を1つ減らし、0になるとエントリが解放されます。

実践 → A3.2 3つの表・TD2・3

§5 特殊ファイル：FIFO、デバイス、`/proc`

「すべてはファイル」とは、ディスク上のデータではない多くのものが同じ `open / read / write / close` のインタフェースを使うという意味です。**FIFO**（名前付きパイプ、`mkfifo`）はブロックを持たない inode です：あるプロセスが書いたものを、別のプロセスがカーネルの 64 KiB のバッファを通して読みます。TD2 のチャットが使う規則：

- `open(O_RDONLY)` は誰かが書き込み用に開くまで待ちます（逆も同じ。`O_NONBLOCK` なら、読み手のいない書き手は `ENXIO` を受け取ります）。
- `PIPE_BUF` = 4096 バイト以下の書き込みは**原子的**です：他の書き手の書き込みと混ざりません。それより大きいと混ざることがあります。
- **最後の書き手が閉じると** `read` は 0（ファイル終端）を返します。

- 読み手がまったくいないのに書くと `SIGPIPE` シグナルが発生します（無視していれば `EPIPE` エラー）。
- その他の特殊ファイル：`/dev` のキャラクタデバイスとブロックデバイス（`mknod` で作成）、UNIX ドメインソケット（ユニット 8 と TD8）、そして読まれたときにカーネルが「ファイル」を生成する仮想ファイルシステム `/proc` と `/sys`。

実際の出力

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' | ./fifo_client
chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

実践 → A3.3 FIFO：チャット・TD2・2

§6 ディレクトリの走査

`opendir` はディレクトリを開き、`readdir` はエントリごとに `struct dirent` を1つ返し（最後は `NULL`）、`closedir` で閉じます。エントリはファイルシステム内部の順序（ext4 ではハッシュ索引の順）で届き、`.` と `..` も含みます。`struct dirent` で重要なのは `d_name`、`d_ino`、`d_type` です。`d_type` があれば `stat` を省けますが、ファイルシステムによっては `DT_UNKNOWN` になり、そのときは `lstat` が必要です。木を走査するときは、ディレクトリへのシンボリックリンクは決してたどりません：循環ができるおそれがあるからです。

C tds/td2/myls.c

```
static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue with the
        others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
}
```

実際の出力

```
$ ./mys -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
```

実践 → TD2・4 自分の `ls`

§7 その先へ：ジャーナリング、コピーオンライト、`fsync`

- **ジャーナリング**。ファイルを作ると複数のブロック（ビットマップ、inode、ディレクトリ）が変わります。途中で電源が落ちるとファイルシステムが不整合になります。`ext4` と `XFS` はまず操作をジャーナルに書き、それから適用します：再起動時には保留中だったものを再実行するか破棄すれば十分です。
- **コピーオンライト**。`Btrfs` と `ZFS` はブロックをその場で上書きせず、新しいブロックを書いてポインタを変えます。これでほぼ無償のスナップショットと全ブロックのチェックサムが得られます。
- **ページキャッシュと `fsync`**。`write` はデータがディスクではなくカーネルのキャッシュに入った時点で戻ります。永続性が必要なプログラム（データベース、保存するエディタ）は `fsync` を呼び、名前も永続させるにはディレクトリにも `fsync` します。
- **FUSE**。ファイルシステムをユーザープロセスとして実装できます（`sshfs`、`rclone`）：カーネルが呼び出しを転送します。